
STOCKHOLM SCHOOL OF ECONOMICS

4339 Asset Pricing and Investments

Assignment 3

Evaluating Mutual Fund Performance

Sharpe ratios, information ratios and factor-model alphas, 2002–2026

Florian Radlmeier · Julius Gremminger

October 2026

Contents

Numerical exercise: evaluating four mutual funds	2
1. Data and monthly excess returns	4
1a. Fund prices from Yahoo Finance	4
1b. Risk-free rate and factors from the Kenneth French data library	4
1c. Merge and compute excess returns	5
2. Sharpe ratio and Information ratio	7
Interpretation (Part 2)	8
3. Is there evidence of manager skill? Factor regressions	10
Alpha summary: FF3 vs. FFC4	12
Interpretation (Part 3)	12

Numerical exercise: evaluating four mutual funds

Ticker	Fund
TRBCX	T. Rowe Price Blue Chip Growth
GLCGX	Goldman Sachs Large Cap Growth Insights
STSEX	BlackRock Exchange Portfolio (BlackRock Shares)
DFSVX	DFA U.S. Small Cap Value Portfolio

Sample: monthly returns, January 2002 – July 2026 (last price date 2026-07-01). **Data:** Yahoo Finance (adjusted fund prices) and the Kenneth French data library (Fama–French 3 factors, momentum, risk-free rate).

```
# If yfinance is missing, run once: %pip install yfinance
import io
import re
import zipfile

import numpy as np
import pandas as pd
import requests
import statsmodels.api as sm
import yfinance as yf
from IPython.display import Markdown, display

FUNDS = {
    "TRBCX": "T. Rowe Price Blue Chip Growth",
    "GLCGX": "Goldman Sachs Large Cap Growth Insights",
    "STSEX": "BlackRock Exchange Portfolio",
    "DFSVX": "DFA U.S. Small Cap Value",
}
TICKERS = list(FUNDS)

PRICE_START = "2001-12-01" # one month early, so January 2002 has a return
PRICE_END = "2026-07-02" # yfinance's end date is exclusive -> last bar is 2026-07-01
SAMPLE_START, SAMPLE_END = "2002-01", "2026-07"

FRENCH_URL = "https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/"
ANN = 12 # months per year, used to annualize

def show_table(table, fmt="{:.3f}", title=None):
    # Display a DataFrame as a Markdown table, so it renders cleanly in the notebook and in the PDF.
    # fmt: one format string for all columns, or a dict {column: format}. Strings are shown as is.
    def cell(value, column):
        if isinstance(value, str):
            text = value
        elif pd.isna(value):
            return ""
        else:
            text = (fmt[column] if isinstance(fmt, dict) else fmt).format(value)
        return text.replace("-", "\N{MINUS SIGN}").replace("*", r"\*") # typographic minus, literal
    ↪ stars

    labels = [str(label) for label in table.index]
    headers = [str(column) for column in table.columns]
    rows = [[cell(value, column) for column, value in row.items()] for _, row in table.iterrows()]
    # pandoc sizes wide PDF tables by these dash counts; headers can wrap, so count their longest
    ↪ word
    widths = [max(3, *map(len, labels))]
```

```
for k, header in enumerate(headers):
    widths.append(max(*(len(word) for word in header.split()), *(len(r[k]) for r in rows)))
lines = [
    "| | " + " | ".join(headers) + " |",
    "|" + "-" * widths[0] + "|" + "|".join("-" * (w - 1) + ":" for w in widths[1:]) + "|",
]
lines += [f"| {label} | " + " | ".join(row) + " |" for label, row in zip(labels, rows)]
if title:
    lines.insert(0, f"**{title}**\n")
display(Markdown("\n".join(lines)))
```

1. Data and monthly excess returns

1a. Fund prices from Yahoo Finance

We use **adjusted** closing prices (`auto_adjust=True`). Mutual funds pay out dividends and capital-gain distributions, which make the raw price drop. Adjusted prices add these payouts back, so `pct_change()` gives the **total return**.

Yahoo's monthly bar dated the 1st of month t holds the **closing price at the end of month t** . The December 2001 bar is therefore the starting value for the January 2002 return.

```
prices = yf.download(
    TICKERS,
    start=PRICE_START,
    end=PRICE_END,
    interval="1mo",
    auto_adjust=True,
    progress=False,
) ["Close"] [TICKERS]
prices.index = prices.index.to_period("M")

fund_ret = prices.pct_change().dropna()

print(f"Prices : {prices.index[0]} -> {prices.index[-1]}")
print(f>Returns: {fund_ret.index[0]} -> {fund_ret.index[-1]} ({len(fund_ret)} months)")
show_table(fund_ret.head(), "{:.4f}", title="Monthly fund returns (decimals), first five months")
```

```
Prices : 2001-12 -> 2026-07
Returns: 2002-01 -> 2026-07 (295 months)
```

Monthly fund returns (decimals), first five months

	TRBCX	GLCGX	STSEX	DFSVX
2002-01	-0.0290	-0.0227	-0.0093	0.0154
2002-02	-0.0292	-0.0371	0.0053	-0.0016
2002-03	0.0472	0.0260	0.0292	0.0986
2002-04	-0.0619	-0.0620	-0.0563	0.0453
2002-05	-0.0168	-0.0251	-0.0196	-0.0315

Note: per-date tables vs. per-fund results

The tables in Part 1 are **per date**, with one row per month. The factor table in 1b below holds the monthly returns of the four factors: market (Mkt-RF), size (SMB), value (HML) and momentum (Mom), plus the risk-free rate (RF). These are the **inputs** (the X variables of the regressions). They are the same for all four funds and don't yet say anything about any fund.

The **per-fund** results come in **Part 3**. There we regress each fund's 295 monthly excess returns on these 295 monthly factor returns. Each regression boils the whole time series down to one alpha (α) and one set of factor loadings (β , s , h , m) **per fund**. In those regression tables, each **column is one fund**.

1b. Risk-free rate and factors from the Kenneth French data library

We download the zipped CSV files directly (`pandas_datareader` doesn't work with pandas 3):

- F-F_Research_Data_Factors: Mkt-RF, SMB, HML, RF

- F-F_Momentum_Factor: Mom (the Carhart momentum factor; “FFC” = Fama–French–Carhart)

Each file has a **monthly** block (dates as YYYYMM) followed by an **annual** block. We keep only the monthly rows. The values are in **percent**, so we divide by 100.

```
MONTHLY_ROW = re.compile(r"^s*d{6}\s*,") # a line that starts with YYYYMM

def load_french_monthly(zip_name):
    # Download one Kenneth French CSV zip and return its monthly table in decimals.
    response = requests.get(FRENCH_URL + zip_name, timeout=60)
    response.raise_for_status()
    with zipfile.ZipFile(io.BytesIO(response.content)) as z:
        lines = z.read(z.namelist()[0]).decode("latin-1").splitlines()

        first = next(i for i, line in enumerate(lines) if MONTHLY_ROW.match(line))
        columns = [c.strip() for c in lines[first - 1].split(",")[1:]] # header sits right above

        rows = []
        for line in lines[first:]:
            if not MONTHLY_ROW.match(line):
                break # the monthly block ends where the annual block starts
            date, *values = [v.strip() for v in line.split(",")]
            rows.append([date, *map(float, values)])

        table = pd.DataFrame(rows, columns=["date", *columns])
        table.index = pd.PeriodIndex(table.pop("date"), freq="M")
        return table / 100

ff3 = load_french_monthly("F-F_Research_Data_Factors_CSV.zip") # Mkt-RF, SMB, HML, RF
mom = load_french_monthly("F-F_Momentum_Factor_CSV.zip")      # Mom
factors = ff3.join(mom, how="inner")
show_table(
    factors.loc[SAMPLE_START:SAMPLE_END].tail(),
    "{:.4f}",
    title="Factor returns and risk-free rate (decimals), last five months of the sample",
)
```

Factor returns and risk-free rate (decimals), last five months of the sample

	Mkt-RF	SMB	HML	RF	Mom
2026-03	-0.0518	0.0044	0.0329	0.0029	0.0156
2026-04	0.0995	0.0019	-0.0137	0.0029	0.0951
2026-05	0.0491	-0.0159	-0.0231	0.0031	0.0139
2026-06	-0.0107	0.0339	0.0358	0.0029	0.0462
2026-07	-0.0061	-0.0192	0.0211	0.0033	-0.1220

1c. Merge and compute excess returns

$$r_{i,t}^e = r_{i,t} - r_{f,t}$$

The market's excess return is French's Mkt-RF (value-weighted CRSP market minus the T-bill rate).

```
data = fund_ret.join(factors, how="inner").loc[SAMPLE_START:SAMPLE_END]
```

```

expected_months = len(pd.period_range(SAMPLE_START, SAMPLE_END, freq="M"))
assert len(data) == expected_months, "missing months after merging"
assert not data.isna().any().any(), "missing values in the merged data"

excess = data[TICKERS].sub(data["RF"], axis=0) # fund excess returns
mkt_excess = data["Mkt-RF"]                  # market excess return

print(f"Sample: {data.index[0]} -> {data.index[-1]} ({len(data)} months)")
show_table(excess.head(), "{:.4f}", title="Monthly excess returns (decimals), first five months")

```

Sample: 2002-01 -> 2026-07 (295 months)

Monthly excess returns (decimals), first five months

	TRBCX	GLCGX	STSEX	DFSVX
2002-01	-0.0304	-0.0241	-0.0107	0.0140
2002-02	-0.0305	-0.0384	0.0040	-0.0029
2002-03	0.0459	0.0247	0.0279	0.0973
2002-04	-0.0634	-0.0635	-0.0578	0.0438
2002-05	-0.0182	-0.0265	-0.0210	-0.0329

```

# Quick look: annualized mean and volatility of the monthly excess returns (in %)
quick_look = pd.DataFrame(
    {
        "Mean (%/yr)": excess.mean() * ANN * 100,
        "Volatility (%/yr)": excess.std() * np.sqrt(ANN) * 100,
        "Worst month (%)": excess.min() * 100,
        "Best month (%)": excess.max() * 100,
    }
)
show_table(quick_look, "{:.2f}", title="Excess returns, January 2002 to July 2026")

```

Excess returns, January 2002 to July 2026

	Mean (%/yr)	Volatility (%/yr)	Worst month (%)	Best month (%)
TRBCX	9.85	17.14	-18.85	14.99
GLCGX	8.53	16.35	-17.68	15.40
STSEX	7.61	13.27	-12.26	10.79
DFSVX	10.71	21.47	-26.71	19.65

2. Sharpe ratio and Information ratio

Sharpe ratio: excess return per unit of total risk

$$SR_i = \frac{E[r_i^e]}{\sigma(r_i^e)}$$

Information ratio relative to the market: the active return over the market per unit of active risk (the *tracking error*)

$$IR_i = \frac{E[r_i - r_M]}{\sigma(r_i - r_M)}, \quad r_i - r_M = r_i^e - r_M^e$$

Everything is annualized: means $\times 12$, standard deviations $\times \sqrt{12}$, so both ratios get $\times \sqrt{12}$.

As a robustness check we also report the **appraisal ratio**. Some textbooks (e.g. BKM) call this the information ratio:

$$AR_i = \frac{\alpha_i}{\sigma(\varepsilon_i)} \quad \text{from the CAPM regression} \quad r_{i,t}^e = \alpha_i + \beta_i r_{M,t}^e + \varepsilon_{i,t}$$

The tracking-error IR implicitly assumes $\beta = 1$. The appraisal ratio adjusts for the fund's actual market beta.

```
def annualized_ratio(series):
    return series.mean() / series.std() * np.sqrt(ANN)

active = excess.sub(mkt_excess, axis=0) # r_i - r_M

capm_alpha, capm_beta, capm_resid_vol = {}, {}, {}
for fund in TICKERS:
    capm = sm.OLS(excess[fund], sm.add_constant(mkt_excess)).fit()
    capm_alpha[fund] = capm.params["const"]
    capm_beta[fund] = capm.params["Mkt-RF"]
    capm_resid_vol[fund] = np.sqrt(capm.mse_resid)
capm_alpha, capm_beta, capm_resid_vol = map(pd.Series, (capm_alpha, capm_beta, capm_resid_vol))

sharpe_table = pd.DataFrame(
    {
        "Mean excess (%/yr)": excess.mean() * ANN * 100,
        "Volatility (%/yr)": excess.std() * np.sqrt(ANN) * 100,
        "Sharpe ratio": excess.apply(annualized_ratio),
    }
)
sharpe_table.loc["Market"] = [
    mkt_excess.mean() * ANN * 100,
    mkt_excess.std() * np.sqrt(ANN) * 100,
    annualized_ratio(mkt_excess),
]

ir_table = pd.DataFrame(
    {
        "Active return (%/yr)": active.mean() * ANN * 100,
        "Tracking error (%/yr)": active.std() * np.sqrt(ANN) * 100,
        "Information ratio": active.apply(annualized_ratio),
        "CAPM beta": capm_beta,
        "CAPM alpha (%/yr)": capm_alpha * ANN * 100,
        "Appraisal ratio": capm_alpha / capm_resid_vol * np.sqrt(ANN),
    }
)
```

```

)

show_table(
  sharpe_table,
  {"Mean excess (%/yr)": "{:.2f}", "Volatility (%/yr)": "{:.2f}", "Sharpe ratio": "{:.3f}"},
  title="Sharpe ratios (annualized)",
)

show_table(
  ir_table,
  {column: "{:.2f}" if "%" in column else "{:.3f}" for column in ir_table.columns},
  title="Information ratio relative to the market and CAPM appraisal ratio (annualized)",
)

```

Sharpe ratios (annualized)

	Mean excess (%/yr)	Volatility (%/yr)	Sharpe ratio
TRBCX	9.85	17.14	0.575
GLCGX	8.53	16.35	0.522
STSEX	7.61	13.27	0.574
DFSVX	10.71	21.47	0.499
Market	9.31	15.27	0.610

Information ratio relative to the market and CAPM appraisal ratio (annualized)

	Active return (%/yr)	Tracking error (%/yr)	Information ratio	CAPM beta	CAPM alpha (%/yr)	Appraisal ratio
TRBCX	0.54	6.05	0.089	1.051	0.06	0.010
GLCGX	-0.78	4.50	-0.173	1.029	-1.05	-0.234
STSEX	-1.70	5.88	-0.288	0.803	0.14	0.027
DFSVX	1.40	11.28	0.124	1.215	-0.60	-0.056

Interpretation (Part 2)

Sharpe ratios. No fund beats the market on a total-risk basis. The market's Sharpe ratio is 0.61. The funds range from 0.50 (DFSVX) to 0.58 (TRBCX). DFSVX has the highest average excess return (10.7%/yr) but also by far the highest volatility (21.5%/yr), so it has the *lowest* Sharpe ratio.

Information ratios (vs. the market). All four are small in absolute value ($|IR| < 0.3$; an IR around 0.5 is usually considered very good):

- **DFSVX (+0.12)** and **TRBCX (+0.09)** beat the market on average (+1.4% and +0.5% per year). Both carry large tracking errors, especially DFSVX at 11.3%/yr, because a small-value fund looks very different from the market index.
- **GLCGX (-0.17)** and **STSEX (-0.29)** trail the market.

Why the appraisal ratio tells a different story. The tracking-error IR treats every deviation from the market as "active", including simply having a market beta different from 1. Using the CAPM, the active return splits exactly into $\bar{r}_i - \bar{r}_M = \alpha_i + (\beta_i - 1) \bar{r}_M^e$:

- DFSVX has $\beta \approx 1.2$. Its higher return is compensation for extra market risk, and its CAPM alpha is actually *negative* (-0.6%/yr).
- STSEX has $\beta \approx 0.8$. It trails the market mainly because it takes less market risk. Its CAPM alpha is slightly *positive* (+0.1%/yr).

- All appraisal ratios are close to zero (-0.23 to $+0.03$). After adjusting for market beta, no fund earns a meaningful abnormal return. Part 3 checks whether this still holds once we also control for size, value and momentum exposure.

3. Is there evidence of manager skill? Factor regressions

We regress each fund's monthly excess return on the factors:

Fama–French 3-factor (FF3):

$$r_{i,t}^e = \alpha_i + \beta_i (r_M - r_f)_t + s_i SMB_t + h_i HML_t + \varepsilon_{i,t}$$

Fama–French–Carhart 4-factor (FFC4):

$$r_{i,t}^e = \alpha_i + \beta_i (r_M - r_f)_t + s_i SMB_t + h_i HML_t + m_i MOM_t + \varepsilon_{i,t}$$

The factor loadings capture the return the fund earns from its *style* (market, size, value, momentum), which an investor could get cheaply from passive factor portfolios. **Skill shows up in the intercept α .** It is the average return the factors cannot explain.

- **Test:** $H_0 : \alpha_i = 0$ vs. $H_1 : \alpha_i \neq 0$. Evidence of skill requires $\alpha > 0$ and statistical significance ($|t| > 1.96$ at the 5% level).
- **Standard errors:** Newey–West (HAC), robust to heteroskedasticity and autocorrelation. The lag length follows the rule of thumb $\lfloor 4(T/100)^{2/9} \rfloor$. Plain OLS t-statistics are shown for comparison.
- Significance stars: * 10%, ** 5%, *** 1%.

```
MODELS = {
    "FF3": ["Mkt-RF", "SMB", "HML"],
    "FFC4": ["Mkt-RF", "SMB", "HML", "Mom"],
}
T = len(data)
NW_LAGS = int(np.floor(4 * (T / 100) ** (2 / 9)))
print(f"T = {T} months, Newey-West lags = {NW_LAGS}")

def fit_factor_model(fund, factor_cols, hac=True):
    X = sm.add_constant(data[factor_cols])
    model = sm.OLS(excess[fund], X)
    if hac:
        return model.fit(cov_type="HAC", cov_kwds={"maxlags": NW_LAGS})
    return model.fit()

fits = {
    (model, fund): fit_factor_model(fund, cols)
    for model, cols in MODELS.items()
    for fund in TICKERS
}
ols_fits = {
    (model, fund): fit_factor_model(fund, cols, hac=False)
    for model, cols in MODELS.items()
    for fund in TICKERS
}
```

T = 295 months, Newey-West lags = 5

```
LABELS = {
    "const": "\alpha (% per month)",
    "Mkt-RF": "\beta (MKT)",
    "SMB": "s (SMB)",
    "HML": "h (HML)",
```

```

    "Mom": "m (MOM)",
}

def stars(p):
    return "***" if p < 0.01 else "**" if p < 0.05 else "*" if p < 0.10 else ""

def regression_table(model):
    # Coefficients with significance stars, Newey-West t-statistics in parentheses below.
    terms = ["const", *MODELS[model]]
    index = [label for term in terms for label in (LABELS[term], "")] + ["adj. R2", "N"]
    columns = {}
    for fund in TICKERS:
        res = fits[(model, fund)]
        cells = []
        for term in terms:
            scale = 100 if term == "const" else 1 # alpha in % per month
            coef = f"{res.params[term] * scale:.3f}{stars(res.pvalues[term])}"
            cells += [coef, f"({res.tvalues[term]:.2f})"]
        cells += [f"{res.rsquared_adj:.3f}", f"{int(res.nobs)}"]
        columns[fund] = cells
    return pd.DataFrame(columns, index=index)

show_table(
    regression_table("FF3"),
    title="Fama-French 3-factor model (Newey-West t-statistics in parentheses)",
)

```

Fama-French 3-factor model (Newey-West t-statistics in parentheses)

	TRBCX	GLCGX	STSEX	DFSVX
α (% per month)	0.003 (0.04)	-0.090 (-1.61)	-0.011 (-0.18)	0.005 (0.09)
β (MKT)	1.090*** (57.20)	1.058*** (69.81)	0.847*** (38.01)	1.029*** (62.87)
s (SMB)	-0.081*** (-2.92)	-0.068** (-2.51)	-0.289*** (-8.44)	0.848*** (23.92)
h (HML)	-0.395*** (-14.69)	-0.272*** (-11.68)	0.100*** (3.71)	0.636*** (26.32)
adj. R ²	0.940	0.958	0.890	0.974
N	295	295	295	295

```

show_table(
    regression_table("FFC4"),
    title="Fama-French-Carhart 4-factor model (Newey-West t-statistics in parentheses)",
)

```

Fama-French-Carhart 4-factor model (Newey-West t-statistics in parentheses)

	TRBCX	GLCGX	STSEX	DFSVX
α (% per month)	−0.003 (−0.05)	−0.106* (−1.87)	0.010 (0.16)	0.012 (0.21)
β (MKT)	1.095*** (53.55)	1.071*** (77.80)	0.832*** (29.78)	1.024*** (54.91)
s (SMB)	−0.080*** (−2.95)	−0.066** (−2.42)	−0.291*** (−8.28)	0.847*** (23.45)
h (HML)	−0.391*** (−12.59)	−0.261*** (−10.37)	0.085*** (3.09)	0.632*** (26.26)
m (MOM)	0.014 (0.56)	0.035* (1.89)	−0.044 (−1.34)	−0.014 (−0.77)
adj. R ²	0.940	0.959	0.892	0.974
N	295	295	295	295

Alpha summary: FF3 vs. FFC4

```

rows = {}
for fund in TICKERS:
    row = {}
    for model in MODELS:
        nw, ols = fits[(model, fund)], ols_fits[(model, fund)]
        row[f"{model}  $\alpha$  (%/yr)"] = nw.params["const"] * ANN * 100
        row[f"{model} t (NW)"] = nw.tvalues["const"]
        row[f"{model} t (OLS)"] = ols.tvalues["const"]
        row[f"{model} p (NW)"] = nw.pvalues["const"]
    rows[fund] = row
alpha_summary = pd.DataFrame.from_dict(rows, orient="index")

show_table(
    alpha_summary,
    "{:.2f}",
    title="Annualized alphas with Newey-West and plain OLS t-statistics",
)

```

Annualized alphas with Newey-West and plain OLS t-statistics

	FF3 α (%/yr)	FF3 t (NW)	FF3 t (OLS)	FF3 p (NW)	FFC4 α (%/yr)	FFC4 t (NW)	FFC4 t (OLS)	FFC4 p (NW)
TRBCX	0.04	0.04	0.05	0.96	−0.04	−0.05	−0.05	0.96
GLCGX	−1.08	−1.61	−1.57	0.11	−1.28	−1.87	−1.86	0.06
STSEX	−0.13	−0.18	−0.14	0.86	0.12	0.16	0.13	0.87
DFSVX	0.07	0.09	0.09	0.92	0.14	0.21	0.20	0.84

Interpretation (Part 3)

FF3 model: no evidence of skill. None of the four alphas is statistically different from zero:

Fund	α (%/yr)	t (NW)	Verdict
TRBCX	+0.04	0.04	essentially zero
GLCGX	−1.08	−1.61	negative, not significant (p = 0.11)
STSEX	−0.13	−0.18	essentially zero
DFSVX	+0.07	0.09	essentially zero

FFC4 model: same conclusion. Adding momentum barely changes anything. The alphas move by at most 0.25 %-points per year, and the adjusted R^2 values are essentially unchanged. Still no fund has a significantly **positive** alpha. The only change is that **GLCGX's** alpha becomes slightly more negative ($-1.28\% / \text{yr}$, $t = -1.87$, $p = 0.06$), which is significant at the 10% level only. GLCGX has a small positive momentum loading ($m = 0.035$, significant at 10%). Part of its return is therefore explained by momentum exposure, which leaves even less for the manager. This is weak evidence of **underperformance**, not skill.

The factor loadings match the fund styles:

- **TRBCX and GLCGX** (large-cap growth): β slightly above 1, negative SMB (large firms), strongly negative HML (growth stocks).
- **STSEX** (large-cap blend): $\beta \approx 0.85$, strongly negative SMB (mega-caps), small positive HML.
- **DFSVX** (small-cap value): large positive SMB (0.85) and HML (0.64), exactly as the name says.
- Momentum loadings are small and not significant at 5% for any fund. That's why FF3 and FFC4 give almost the same alphas.

The factors explain almost everything. The adjusted R^2 is 0.89–0.97. These funds' returns are mostly their factor exposures, which an investor could replicate cheaply with passive factor/index portfolios. DFSVX ($R^2 = 0.974$) is essentially a systematic size-and-value tilt. Its positive information ratio in Part 2 came from SMB/HML exposure, not stock picking. Once we control for it, its alpha is about zero.

Conclusion. Neither the FF3 nor the FFC4 model shows evidence of manager skill (a significantly positive α) for any of the four funds over 2002–2026. Two caveats:

- Fund returns on Yahoo are **net of fees**. An alpha of about zero after fees means that, at best, the managers generate just enough gross alpha to cover their fees, so investors capture none of it (consistent with Berk & Green, 2004).
- We test four funds at once. With a multiple-testing correction, the stricter significance thresholds (e.g. $|t| > 3$ as proposed by Harvey, Liu & Zhu, 2016) make the evidence for any skill even weaker.